Coercive Subtyping for Implicit Functorial Programming

Programming with typeclass-defined combinators like fmap allows programs a great deal of generality and concision, but programmers have to explicitly invoke combinators in the right places in order to get their programs to type check. In this paper, we propose inferring these operations using a coercive subtyping discipline. It infers where to insert operations by looking for points in the program where types do not yet agree but can be made to line up by adding coercions. For functors, we promote the fmap combinator $\texttt{fmap} : (A \to B) \to F\,A \to F\,B$ to a coercion $\texttt{fmap} : A \to B \leq F\,A \to F\,B$. We incorporate it into a monomorphic system of standard coercions for base types, arrow types, and functor types. We prove that the resulting system of coercions is coherent, meaning that all possible choices of coercions from A to B end up equivalent. The proof of the theorem is based on coherence by normalization, an approach to coherence proofs that constitutes an application of normalization by evaluation. All definitions and theorems in the paper are proved in Agda, which played a valuable role in the design of the system.

Additional Key Words and Phrases: subtyping, functors, normalization by evaluation

ACM Reference Format:
. 2026. Coercive Subtyping for Implicit Functorial Programming. *Proc. ACM Program. Lang.* 1, 1, Article 1 (January 2026), 13 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Typeclass abstractions like Functor, Applicative, and Monad are essential to modern functional programming. The liberal application of type class combinators allows concise generic programming even in the presence of complex effects and data structures. For example, consider writing a Haskell program that increments integers wrapped in a Maybe type.

```
incrMaybe :: Maybe Int -> Maybe Int
incrMaybe (Just x) = Just (succ x)
incrMaybe Nothing  = Nothing
```

This naive definition of incrMaybe has to unwrap its argument with a pattern match and then wrap its output back up. A more idiomatic definition uses <$>, the fmap combinator, which handles the unwrapping and wrapping. This has the added benefit of being polymorphic, as it will work for any other instance of the Functor typeclass.

```
incrMaybe :: Maybe Int -> Maybe Int
incrMaybe x = succ <$> x
```

In the presence of sufficient type information, we are interested in further reducing the boilerplate involved by making fmap completely implicit. This would permit implementing incrMaybe just as one would want to write it, with no use of fmap at all.

```
incrMaybe :: Maybe Int -> Maybe Int
incrMaybe x = succ x
```

Given the declared type, there is really only one way to make this program make sense. It does not typecheck otherwise, so what would be the harm in inserting an fmap? Automatically lifting functions to operate on arrays and other data structures is already prevalent both in programming languages and informal mathematical notation. If programming with functors can be handled in this way then implicit programming with more powerful abstractions like applicative functors [5] and monads could also become tractable.

Author's Contact Information:

2026. ACM 2475-1421/2026/1-ART1
https://doi.org/XXXXXXX.XXXXXXX

To insert missing fmaps where they are needed, we propose a discipline of implicit coercions, which are meant to be inserted during a combined type inference and elaboration pass. An implementation would walk the provided term, send a collection of subtyping constraints to a solver, and then use the solver's response to decide where to put coercions.

In this paper, we address the problem of inserting implicit typeclass operations for the Functor typeclass. This allows applying pure functions to data stored in functors without having to insert calls to fmap. We describe a combined coercion and subtyping judgment

$$c : A \leq B$$

where the coercion c is drawn from a simple combinator calculus. The coercions elaborate into terms in a functional programming language, but come from a much simpler combinator calculus. We add a special coercion

$$\mathsf{fmap} : A \to B \leq F\,A \to F\,B$$

which lifts its argument into the functor. The other coercions are standard and include, for instance, the mixed-variance coercion

$$\mathsf{arr}\ c_1\ c_2 : A \to B \leq U \to V$$

that turns two coercions $c_1 : U \leq A$ and $c_2 : B \leq V$ into a coercion between the appropriate arrow types.

The system of coercions is declarative, in the sense that it is a good specification of what coercions ought to be available in the system, but its derivations are nondeterministic. This presents two problems: a nondeterministic subtyping judgment is not suitable for implementation and the coercions inserted by two different determinizations may end up doing different things to the program.

We solve both problems. First, we define an algorithmic subtyping system with unique derivations. It is proved sound and complete with respect to the declarative subtyping system. Second, in order to get some assurance that this system of coercions is safe, we give a proof of its coherence. Coherence, in this context, says that any two coercions $c_1, c_2 : A \leq B$ are semantically equivalent, in a sense to be made precise later. Our proof of coherence exploits the theory of normalization by evaluation, in which a complicated equivalence is analyzed by identifying canonical forms for its equivalence classes. In this case, the canonical forms are precisely the coercions from the algorithmic subtyping judgment.

Our paper makes the following contributions.

- Declarative and algorithmic coercive subtyping judgments for inferring missing uses of fmap in functional programs.
- An equivalence-preserving translation of coercion combinators into lambda calculus.
- A proof of soundness, completeness, and uniqueness for algorithmic coercions.
- A proof of coherence for declarative subtyping based on normalization by evaluation.
- A mechanization of all of the above in Agda.[1]

2 Overview

The types and subtyping rules at issue are shown in Figure 1, which shows a base type, arrow types, and functor types, along with the usual subtyping rules for all of these and the new Fmap rule. This is the declarative system. Everything is standard except Fmap, which permits using functions $A \to B$ at the lifted type $F\,A \to F\,B$. This rule may be iterated, permitting the use of a function at $F\,A \to F\,B$, $F\,F\,A \to F\,F\,B$, and so on. These rules serve as a good specification of the system, but they have practical and theoretical limitations.

[1]References to Agda definitions are rendered in `teal`. The file is available in the supplementary material.

$$\boxed{A \leq_{\mathsf{n}} B}$$

$$A ::= \mathbb{B} \mid A \to A \mid F\,A$$

$$\textsc{Refl}\ \frac{}{A \leq A} \qquad \textsc{Trans}\ \frac{A \leq B \qquad B \leq C}{A \leq C} \qquad \textsc{Arr}\ \frac{U \leq A \qquad B \leq V}{A \to B \leq U \to V}$$

$$\textsc{Mono}\ \frac{A \leq B}{F\,A \leq F\,B} \qquad \textsc{Fmap}\ \frac{}{A \to B \leq F\,A \to F\,B}$$

Fig. 1. Types and subtype relationships for a calculus with implicit fmap.

The declarative rules are not suitable for an implementation because they are not deterministic: too many rules apply to possible subtyping queries. Besides the fact that it is always possible to appeal to the Trans rule, queries of the form $A \to B \leq F\,U \to F\,V$ must choose between applying Arr and Fmap. The way the Fmap rule overlaps with the arrow rule is the chief source of technical complications in the sequel.

Declarative coercions (Fig. 2) are nothing more than proof terms for the declarative subtyping judgment. Playing fast and loose with Curry-Howard, we will often refer to the "type" of a coercion: this refers to the subtyping judgment $A \leq B$, which is not a type in the object language. However, we do provide a translation of coercions into the object language, and under this translation $A \leq B$ does become a type of functions from A to B.

The ambiguity in subtyping derivations discussed above means that for a fixed judgment $A \leq B$ there may be many possible coercions c_1, c_2, and so on. In the case of transitivity, the following pair of derivations raises the question of associativity of composition.

$$\frac{\dfrac{c_1 : A \leq B \qquad c_2 : B \leq C}{c_1 \cdot c_2 : A \leq C} \qquad c_3 : C \leq D}{(c_1 \cdot c_2) \cdot c_3 : A \leq D} \qquad \frac{c_1 : A \leq B \qquad \dfrac{c_2 : B \leq C \qquad c_3 : C \leq D}{c_2 \cdot c_3 : B \leq D}}{c_1 \cdot (c_2 \cdot c_3) : A \leq D}$$

The ambiguity of the arrow rules leads to confusion between fmap and arr. Together with composition, this produces another pair of possibly inequivalent derivations. The first uses fmap after arr:

$$\frac{\dfrac{c_1 : U \leq A \qquad c_2 : B \leq V}{\mathsf{arr}\ c_1\ c_2 : A \to B \leq U \to V} \qquad \dfrac{}{\mathsf{fmap} : U \to V \leq F\,U \to F\,V}}{\mathsf{arr}\ c_1\ c_2 \cdot \mathsf{fmap} : A \to B \leq F\,U \to F\,V}$$

The second uses fmap before arr:

$$\frac{\dfrac{}{\mathsf{fmap} : A \to B \leq F\,A \to F\,B} \qquad \dfrac{\dfrac{c_1 : U \leq A}{\mathsf{mono}\ c_1 : F\,U \leq F\,A} \qquad \dfrac{c_2 : B \leq V}{\mathsf{mono}\ c_2 : F\,B \leq F\,V}}{\mathsf{arr}\ (\mathsf{mono}\ c_1)\ (\mathsf{mono}\ c_2) : F\,A \to F\,B \leq F\,U \to F\,V}}{\mathsf{fmap} \cdot \mathsf{arr}\ (\mathsf{mono}\ c_1)\ (\mathsf{mono}\ c_2) : A \to B \leq F\,U \to F\,V}$$

Keeping these examples in mind, we now discuss the implications of ambiguity in subtyping derivations. Under a coercive subtyping discipline, type inference becomes an elaboration pass that walks an untyped term e, inserting coercions as it goes, to construct a typed term e'. In theoretical terms, the type system has a judgment shaped like $\Gamma \vdash e \rightsquigarrow e' : A$ where e is an input and e' is an output. It appeals to a subtyping judgment $c : A \leq B$ that emits a coercion c, which is brought

$\boxed{c : A \leq B}$

$$\textsc{CId}\ \frac{}{\mathsf{id}_A : A \leq_{\mathsf{n}} A} \qquad \textsc{CComp}\ \frac{c_1 : A \leq B \qquad c_2 : B \leq C}{c_1 \cdot c_2 : A \leq C}$$

$$\textsc{CArr}\ \frac{c_1 : U \leq A \qquad c_2 : B \leq V}{\mathsf{arr}\ c_1\ c_2 : A \to B \leq U \to V}$$

$$\textsc{CMono}\ \frac{c : A \leq B}{\mathsf{mono}\ c : F\,A \leq F\,B} \qquad \textsc{CFmap}\ \frac{}{\mathsf{fmap} : A \to B \leq F\,A \to F\,B}$$

Fig. 2. Declarative coercions.

back into the grammar of terms by a translation $\mathsf{C}(c) : A \to B$. This is all tied together in the subsumption rule:

$$\textsc{Sub}\ \frac{\Gamma \vdash e : A \qquad c : A \leq B}{\Gamma \vdash e \rightsquigarrow \mathsf{C}(c)\ e : B}$$

This process may alter the semantics of e, if they are even defined at all. But there is another problem. In a declarative type system there are, in general, many derivations of the same typing judgment. When dealing with coercive subtyping, this means that while there is generally a unique derivation for a fixed pair of e and e', there may be many possible choices of e' from the same initial e. This means that different implementations of the same type system could give different semantics to the same program.

Coherence for subtyping and elaboration. To ensure the meaning of a program is stable across all possible typing derivations, metatheorists prove *coherence*. A subtyping judgment is coherent when all coercions $c : A \leq B$ between the same types are semantically equivalent. This means that every path in the "coercion graph" is equivalent or, to borrow a category-theoretic slogan, "all diagrams commute." The elaboration judgment is coherent when all e' with $\Gamma \vdash e \rightsquigarrow e' : B$ are semantically equivalent and furthermore e is semantically equivalent to e'. Of course, nobody ever runs code that was elaborated by a declarative type system. Instead, implementations of type inference will hew closely to an algorithmic type system including a system of coercions $c_a : A \leq_{\mathsf{a}} B$. By virtue of being algorithmic, this system will have unique coercions and emit unique elaborated terms. It enjoys subtyping coherence automatically because its coercions and elaborated terms are not merely equivalent, but equal.

This is not a shortcut to solving the coherence problem. However expedient it may be to determinize everything in sight, it will end up costing more in the long term. There are many perfectly reasonable algorithmic typing judgments (and implementations) associated to any one declarative system. A Damas-Hindley-Milner system will produce different derivations than one based on bidirectional typing, and so on. A future algorithmic type system with its own idiosyncratic solution to the problem of determinizing the rules will include its own typed grammar of coercions $c_b : A \leq_{\mathsf{b}} B$ and its own elaboration judgment. In order to show these have anything to do with the other coercions $c_a : A \leq_{\mathsf{a}} B$, we need a common semantics in which to compare them. Without a declarative system, our only option is try and prove a contextual equivalence

$$\mathsf{C}_{\mathsf{a}}(c_a) \approx \mathsf{C}_{\mathsf{b}}(c_b).$$

Needless to say, this is not a direct route. Since the invariants the two algorithmic type systems exploit in order to determinize the rules of the declarative system are likely to be different, the coercions they produce for a given query are unlikely to share any structural similarities. On top of that, equational reasoning in the target language may be complex or underspecified when compared to equational reasoning about coercions, which ought to be terminating and effect-free programs.

Coherence by normalization: a recipe. We propose coherence by normalization, a formulation of coherence for subtyping that can exploit the benefits of choosing a distinguished algorithmic system without requiring future algorithmic systems to be related directly to it. Rather than defining coercions only for a deterministic judgment, we begin from a grammar of coercions $c : A \leq B$ for the declarative system. These combinators are equipped with an axiomatized equivalence relation $c_1 \simeq c_2$. The grammar will supply proof terms for the judgment $A \leq B$ without appealing to the grammar of the programming language. The equivalence relation will be finitely axiomatized, in the style of abstract algebra, and will not appeal to the translation $\mathtt{C}$ of coercions into expressions. In order to justify the equivalence relation, we check that the translation $\mathtt{C}$ is sound:

$$c_1 \simeq c_2 \Longrightarrow \mathtt{C}(c_1) \approx \mathtt{C}(c_2)$$

We then define an algorithmic system $n : A \leq_{\mathsf{n}} B$ of *normal coercions*, so named because they will be used to put (declarative) coercions into normal form. These will enjoy unique derivations. In order to prove coherence, we begin by saying how to translate coercions into normal form and back:

$$\mathsf{eval} : A \leq B \to A \leq_{\mathsf{n}} B$$
$$\mathsf{reify} : A \leq_{\mathsf{n}} B \to A \leq B$$

That eval and reify can be defined at all constitutes proof that the normal system is sound and complete with respect to the declarative system. The next step in normalization by evaluation is to prove that $\mathsf{reify}(\mathsf{eval}\ c) \simeq c$. By exhibiting a left inverse, we show that eval is injective up to equivalence. This implies coherence, since we know $\mathsf{eval}\ c_1 = \mathsf{eval}\ c_2$ from the uniqueness theorem for normal coercions.

Extending the recipe with a new implementation. Consider the situation now for a future algorithmic judgment $c_a : A \leq_{\mathsf{a}} B$. We already know declarative coercions are coherent, so we just need to know that in the new system of coercions each c_a is equivalent to a declarative coercion c. All we have to do is define another pair of functions

$$\mathsf{eval}_{\mathsf{a}} : A \leq B \to A \leq_{\mathsf{a}} B$$
$$\mathsf{reify}_{\mathsf{a}} : A \leq_{\mathsf{a}} B \to A \leq B,$$

This time around, the equivalence $\mathsf{reify}_{\mathsf{a}}\ (\mathsf{eval}_{\mathsf{a}}\ c) \equiv c$ follows from the coherence result that was already proved. It does have to be checked that $\mathtt{C}_{\mathsf{a}}(c_a) \approx \mathtt{C}(\mathsf{reify}_{\mathsf{a}}\, c_a)$, but this can always be taken as a definition of $\mathtt{C}_{\mathsf{a}}(c_a)$.

Comparison to Reynolds' method. The coherence problem was studied as early as 1980 by Reynolds [6], who outlined an algebraic approach to coherence of elaboration that complements our own. It was Reynolds who recognized that coercions form a *category*: there are types $A, B, \ldots$ and between any two types there is a set of coercions $\mathsf{Coe}(A, B)$. There should always be identity coercions $\mathsf{id}_A \in \mathsf{Coe}(A, A)$, which do nothing, and an associative and unital composition operation $\circ : \mathsf{Coe}(B, C) \times \mathsf{Coe}(A, B) \to \mathsf{Coe}(A, C)$. Reynolds assumed that coercions form a preorder, that is, for every pair of arrows $c1, c2 \in \mathsf{Coe}(A, B)$, $c1 = c2$. While this is true (up to quotienting by

$\boxed{n : A \leq_{\mathsf{n}} B}$ $\boxed{na : A, B \leq_{\mathsf{na}} U, V}$

$$\textsc{NId}\ \frac{}{\mathsf{nid} : \mathbb{B} \leq_{\mathsf{n}} \mathbb{B}} \qquad \textsc{NMono}\ \frac{n : A \leq_{\mathsf{n}} B}{\mathsf{nmono}\ n : F\,A \leq_{\mathsf{n}} F\,B}$$

$$\textsc{NArr}\ \frac{na : A, B \leq_{\mathsf{na}} U, V}{\mathsf{narr}\ na : A \to B \leq_{\mathsf{n}} F\,A \to F\,B} \qquad \textsc{NAFmap}\ \frac{na : A, B \leq_{\mathsf{na}} U, V}{\mathsf{nfmap}\ na : A, B \leq_{\mathsf{na}} F\,U, F\,V}$$

$$\textsc{NAArr}\ \frac{n_1 : U \leq_{\mathsf{n}} A \qquad n_2 : B \leq_{\mathsf{n}} V}{\mathsf{naarr}\ n_1\ n_2 : A, B \leq_{\mathsf{na}} U, V}$$

Fig. 3. Normalized coercions.

equivalence) for any coherent coercive subtyping relation, our approach to coherence is a robust technique for establishing it in the first place.

3 Defining and justifying the equational theory

The first order of business in establishing coherence is to decide on a reasonable definition of equivalence for coercions. As mentioned above, we take an algebraic approach and define an equivalence using a short list of axioms. It helps that coercion combinators appear to form a category, so the appropriate axioms are, for the most part, easy to guess.

Definition 1 (Coercion congruence). *The equivalence relation $\simeq$ is the least congruence obeying the following laws, which are implicitly universally quantified.*

$$\begin{aligned}
\mathsf{id} \cdot c &\simeq c & \mathsf{mono}\ (c_1 \cdot c_2) &\simeq \mathsf{mono}\ c_1 \cdot \mathsf{mono}\ c_2 \\
c \cdot \mathsf{id} &\simeq c & \mathsf{arr}\ \mathsf{id}_A\ \mathsf{id}_B &\simeq \mathsf{id}_{A \to B} \\
c_1 \cdot (c_2 \cdot c_3) &\simeq (c_1 \cdot c_2) \cdot c_3 & \mathsf{arr}\ (c_1 \cdot c_2)\ (c_3 \cdot c_4) &\simeq \mathsf{arr}\ c_2\ c_3 \cdot \mathsf{arr}\ c_1\ c_4 \\
\mathsf{mono}\ (\mathsf{id}_A) &\simeq \mathsf{id}_{F\,A} & \mathsf{fmap} \cdot \mathsf{arr}\ (\mathsf{mono}\ c_1)\ (\mathsf{mono}\ c_2) &\simeq \mathsf{arr}\ c_1\ c_2 \cdot \mathsf{fmap}
\end{aligned}$$

In order, the laws say that coercions form a category, mono is a functor on that category, and arr is a bifunctor on that category. The final law, which we refer to as the *special fmap law*, is unusual becuase fmap is not a functor on coercions but instead a functor on types. In the equational theory of coercions, all that can be said about fmap is how it interacts with other coercions on arrow types. No rule is needed to describe how it interacts with fmap, since fmap · fmap is, in general, only equal to itself. The special fmap law implies the last pair of ambigous derivations shown in Section 2 is coherent.

3.1 Compiling coercions correctly

In order to justify the equational theory, especially the new fmap law, and show that these coercion combinators can be soundly compiled to a programming language, we provide a translation into typed lambda calculus equipped with an abstract functor F. Then we prove that the translation preserves equivalence, which is given by the equational theory in the source language and by

$\beta\eta$-equivalence, function extensionality, and fmap laws in the target.

$$\begin{aligned} \mathtt{C}(\mathbb{B}) &= \mathtt{bool} \\ \mathtt{C}(A \to B) &= \mathtt{C}(A) \texttt{ -> } \mathtt{C}(B) \\ \mathtt{C}(F\,A) &= \mathtt{FC}(A) \end{aligned} \qquad \begin{aligned} \mathtt{C}(\mathsf{id}_A) &= \lambda x : \mathtt{C}(A).\, x \\ \mathtt{C}(c_1 \cdot c_2) &= \lambda x.\, \mathtt{C}(c_2)\ (c_1\ x) \\ \mathtt{C}(\mathsf{arr}\ c_1\ c_2) &= \lambda f\, x.\, c_2\ (f\ (c_1\ x)) \\ \mathtt{C}(\mathsf{mono}\ c) &= \mathtt{fmap}\ \mathtt{C}(c) \\ \mathtt{C}(\mathsf{fmap}) &= \mathtt{fmap} \end{aligned}$$

Proposition 2 (interp-sound). *For all $c_1, c_2 : A \le B$,*

$$c_1 \simeq c_2 \implies \mathtt{C}(c_1) \equiv_{\beta\eta\mathsf{F}} \mathtt{C}(c_2)$$

Proof. By induction on $c_1 \simeq c_2$. Extensionality appears necessary in the case for the special fmap law. □

4 Algorithmic judgment and normal coercions

Using the equational theory, we now design the algorithmic system. The idea is to remove the Refl and Trans rules and instead have one rule for each type former. [todo: mult table thing]

4.1 Uniqueness of normal coercions

It is not immediately clear that normal coercions have $n_1 = n_2$ for any $n_1, n_2 : A \le_{\mathsf{n}} B$. The proof of this ought to look like an induction on one paired with some inversion lemmas on the other, but there are issues in the arrow cases. For instance, inverting $na_2 : A, B \le_{\mathsf{na}} F\,U, F\,V$ suggests that na_2 has the form $na_2 = \mathsf{nfmap}\ na'$ or $na_2 = \mathsf{naarr}\ n_1'\ n_2'$, but both possiblities remain plausible. Fortunately, there are some invariants preserved by subtyping that will allow ruling out the latter possibility.

The subtyping relations $A \le_{\mathsf{n}} B$ and $A, B \le_{\mathsf{na}} U, V$ exhibit a predictable relationship between the number of leading functors in the types on their left and right. To this point, define the *peel number* of a type to be $\mathsf{peel}(F\,A) = 1 + \mathsf{peel}(A)$ and $\mathsf{peel}(A) = 0$ otherwise. The peel number is preserved by normal subtyping and increased by arrow subtyping.

Proposition 3 (Peel invariants). *Let A, B, U, and V be types.*

(1) *If $A \le_{\mathsf{n}} B$ then $\mathsf{peel}(A) = \mathsf{peel}(B)$.*
(2) *If $A, B \le_{\mathsf{n}} U, V$ then $\mathsf{peel}(A) + k = \mathsf{peel}(U)$ and $\mathsf{peel}(B) + k = \mathsf{peel}(V)$ for some k.*

Proof. By mutual induction on $\le_{\mathsf{n}}$ and $\le_{\mathsf{na}}$. □

With this in hand, we can distinguish arrow coercions. In fact, the k that appears in the invariant for arrow subtyping counts the number of nfmap constructors that appear in the corresponding derivation.

Proposition 4 (ncoe-uniq). *Let A, B, U, and V be types.*

- *If $n_1 : A \le_{\mathsf{n}} B$ and $n_2 : A \le_{\mathsf{n}} B$ then $n_1 = n_2$.*
- *If $na_1 : A, B \le_{\mathsf{na}} U, V$ and $na_2 : A, B \le_{\mathsf{na}} U, V$ then $na_1 = na_2$.*

Proof. By mutual induction on n and na. We show a couple of representative cases.

- Case $n_1 = \mathsf{nmono}\ n_1'$: By inversion, $n_2 = \mathsf{nmono}\ n_2'$ and we can apply the IH.
- Case $na_1 = \mathsf{nfmap}\ na_1'$ with $na_1' : A, B \le_{\mathsf{na}} U, V$: By Prop. 3, we have $\mathsf{peel}(A) \le \mathsf{peel}(U)$. If $na_2 = \mathsf{nfmap}\ na_2'$ then the IH applies. Otherwise, $na_2 = \mathsf{naarr}\ n_1\ n_2$ for some coercions $n_1 : F\,U \le_{\mathsf{n}} A$ and $n_2 : B \le F\,V$. Applying Prop. 3 again yields $\mathsf{peel}(A) = \mathsf{peel}(U) + 1$, a contradiction. □

5 Coherence

In this section, we prove that all coercions of the same type are equivalent. We establish our result using the methods of normalization by evaluation, which is more commonly used to establish that a language has decidable equality by lifting it from a model with an already decidable relation, which is often closer to syntactic equality. In this section we establish coherence for coercions by lifting it from a proof of uniqueness for normal coercions.

Theorem 5. *If* $c_1, c_2 : A \leq B$ *then* $c_1 \simeq c_2$.

5.1 Reification and evaluation

Normal coercions are immediately interpretable as coercions, as exhibited by the following straightforward definition of reify.

$$\begin{aligned}
\mathsf{reify}\ \mathsf{nid} &= \mathsf{id}_{\mathbb{B}} \\
\mathsf{reify}\ (\mathsf{nmono}\ n) &= \mathsf{mono}\ (\mathsf{reify}\ n) \\
\mathsf{reify}\ (\mathsf{narr}\ na) &= \mathsf{reify}_{\mathsf{na}}\, na \\
\mathsf{reify}_{\mathsf{na}}\ (\mathsf{naarr}\ n_1\ n_2) &= \mathsf{arr}\ (\mathsf{reify}\ n_1)\ (\mathsf{reify}\ n_2) \\
\mathsf{reify}_{\mathsf{na}}\ (\mathsf{nfmap}\ na) &= \mathsf{reify}_{\mathsf{na}}\, na \cdot \mathsf{fmap}
\end{aligned}$$

Note that the reification of nfmap *na* puts fmap after the reification of *na* and not before. This is an arbitrary choice, like the choosing between cons lists and snoc lists. Putting fmap at the front would affect the "bias" of subsequent proofs but all the top-level theorems would say the same things.

Reification is sound because all functions preserve equality. By composition, $[\![-]\!] \circ \mathsf{reify}$ is also sound. This means our syntactic equality agrees with the intended model in lambda calculus. The type of reify is also informative. Read through Curry-Howard, it says that the normal subtyping relation is sound (but not necessarily complete) for subtyping.

$$A \leq_{\mathsf{n}} B \Longrightarrow A \leq B$$

Evaluation, which passes from coercions to normal coercions, relies on some definable operators on normal coercions. This is where most of the work happens.

$$\mathsf{mkid}_A : A \leq_{\mathsf{n}} A \qquad \mathsf{mkcomp} : A \leq_{\mathsf{n}} B \times B \leq_{\mathsf{n}} C \to A \leq_{\mathsf{n}} C$$

These correspond to standard lemmas in the metatheory of subtyping. The first proves a reflexivity rule is admissible, while the second proves a transitivity rule is admissible. Evaluation uses these to establish the completeness of normal subtyping. Promoting these results from theorems to functions makes them proof-relevant, which will ultimately allow us to show a stronger result: implicit coercions in both theories behave the same.

The identity and composition operators permit a structurally recursive definition of eval , which is otherwise defined in terms of constructors. From an algebraic point of view, we are exhibiting an algebraic structure on normal coercions that then induces the interpretation eval from the free algebra—coercions—into normal coercions.

$$\begin{aligned}
\mathsf{eval}\ \mathsf{id}_A &= \mathsf{mkid}_A \\
\mathsf{eval}\ (c_1 \cdot c_2) &= \mathsf{mkcomp}\ (\mathsf{eval}\ c_1)\ (\mathsf{eval}\ c_2) \\
\mathsf{eval}\ (\mathsf{mono}\ c) &= \mathsf{nmono}\ (\mathsf{eval}\ c) \\
\mathsf{eval}\ (\mathsf{arr}\ c_1\ c_2) &= \mathsf{narr}\ (\mathsf{naarr}\ (\mathsf{eval}\ c_1)\ (\mathsf{eval}\ c_2)) \\
\mathsf{eval}\ \mathsf{fmap} &= \mathsf{narr}\ \mathsf{nfmap}
\end{aligned}$$

The identity operator is defined by induction on type. This helps explain why the normal coercions do not have a built-in nid_A operator for any type A, but only include it for base types. Otherwise, at every type A the two distinct proofs nid_A and mkid_A would both prove $A \leq_{\mathsf{n}} A$, which would break uniqueness (Prop. 4).

$$\begin{aligned}\mathsf{mkid}_{\mathbb{B}} &= \mathsf{nid}\\ \mathsf{mkid}_{A\to B} &= \mathsf{narr}\ (\mathsf{naarr}\ (\mathsf{mkid}_A)\ (\mathsf{mkid}_B))\\ \mathsf{mkid}_{F\,A} &= \mathsf{nmono}\ (\mathsf{mkid}_A)\end{aligned}$$

The composition operator is more complex. It deals with the mutually inductive types of normal coercions and arrow coercions. While most clauses in the definition below are straightforward structural recursion, there is a snag in one case, which composes an arrow coercion ending in an fmap with another arrow coercion. The available fmap constructor in the grammar for *na* only puts fmaps on the right, so we need to push the fmap "through" the second argument, all the way to the end, and adjust the coercion to fit.

More precisely, we are defining an operation that composes an arrow coercion *na* with an fmap on the left. We do not actually care what this does to the term as long as it acts like the coercion fmap $\cdot$ reify *na*. It has the type $\mathsf{nfmap}_{\mathsf{l}} : F\,A, F\,B \leq_{\mathsf{n}} U, V \to A, B \leq_{\mathsf{n}} U, V$ and is defined by only two clauses.

$$\begin{aligned}\mathsf{nfmap}_{\mathsf{l}}\ (\mathsf{nfmap}\ na) &= \mathsf{nfmap}\ (\mathsf{nfmap}_{\mathsf{l}} na)\\ \mathsf{nfmap}_{\mathsf{l}}\ (\mathsf{naarr}\ (\mathsf{nmono}\ n_1)\ (\mathsf{nmono}\ n_2)) &= \mathsf{nfmap}\ (\mathsf{naarr}\ n_1\ n_2)\end{aligned}$$

These rules are exhaustive. The trick here is that if *na* is safe to compose with an fmap on the left, it must have F types in its domain(s). If it has the form $\mathsf{naarr}\ n_1'\ n_2'$, the only way the sub-coercions n_1' and n_2' can be well-typed is by beginning with nmono. This procedure, which pushes an fmap through a term by "demoting" nmono coercions, will be justified by the special fmap law.

Let us return to our original task: defining composition for normal coercions. This can now be carried out by a simple structural recursion.

$$\begin{aligned}\mathsf{mkcomp}\ n_1\ \mathsf{nid} &= n_1\\ \mathsf{mkcomp}\ \mathsf{nid}\ n_2 &= n_2\\ \mathsf{mkcomp}\ (\mathsf{nmono}\ n_1)\ (\mathsf{nmono}\ n_2) &= \mathsf{nmono}\ (\mathsf{mkcomp}\ n_1 n_2)\\ \mathsf{mkcomp}\ (\mathsf{narr}\ na_1)\ (\mathsf{narr} na_2) &= \mathsf{narr}\ (\mathsf{mkcomp}_{\mathsf{na}}\ na_1\ na_2)\\ \mathsf{mkcomp}_{\mathsf{na}}\ (\mathsf{naarr}\ n_1\ n_2)\ (\mathsf{naarr}\ n_3\ n_4) &= \mathsf{naarr}\ (\mathsf{mkcomp}\ n_3\ n_1)\ (\mathsf{mkcomp}\ n_2\ n_4)\\ \mathsf{mkcomp}_{\mathsf{na}}\ na_1\ (\mathsf{nfmap}\ na_2) &= \mathsf{nfmap}\ (\mathsf{mkcomp}_{\mathsf{na}}\ na_1\ na_2)\\ \mathsf{mkcomp}_{\mathsf{na}}\ (\mathsf{nfmap}\ na_1)\ na_2 &= \mathsf{mkcomp}_{\mathsf{na}}\ na_1\ (\mathsf{nfmap}_{\mathsf{l}}\ na_2)\end{aligned}$$

We have now fully defined the evaluation function eval . It respects equivalence by virtue of preserving the types of its arguments.

Proposition 6 (`eval-sound`). *For all* $c_1, c_2 : A \leq B$, *if* $c_1 \simeq c_2$ *then* $\mathsf{eval}\ c_1 = \mathsf{eval}\ c_2$.

Proof. Since $\mathsf{eval}\ c_1, \mathsf{eval}\ c_2 : A \leq_{\mathsf{n}} B$, this follows from uniqueness (Prop. 4). □

5.2 Reification inverts evaluation

We now hew close to the path laid by normalization by evaluation and prove that reify is a left inverse to eval. This makes the normalization function reify $\circ$ eval sound. The proof of this lemma exercises all the rules of the equivalence relation $\simeq$ on coercions, including the special fmap rule.

The first order of business in this proof is to establish that reify is a homomorphism for the defined algebra on normal coercions.

Proposition 7 (reify-mkid). *For all A,* reify $\mathsf{mkid}_A \simeq \mathsf{id}_A$.

Proof. By induction on A using the identity laws for mono and arr. □

Proposition 8 (reify-na-fmap-l). *If $na : F\,A, F\,B \leq_{\mathsf{na}} U, V$ then*

$$\mathsf{reify}_{\mathsf{na}}\ (\mathsf{nfmap}_{\mathsf{l}}\ na) \simeq \mathsf{fmap} \cdot \mathsf{reify}_{\mathsf{na}}\ na.$$

Proposition 9 (reify-mkcomp). *Let $A, B, U, V, X,$ and Y be types.*

(1) *For all $n_1 : A \leq_{\mathsf{n}} B$ and $n_2 : B \leq_{\mathsf{n}} C$,*

$$\mathsf{reify}\ (\mathsf{mkcomp}\ n_1\ n_2) \simeq (\mathsf{reify}\ n_1) \cdot (\mathsf{reify}\ n_2).$$

(2) *For all $na_1 : A, B \leq_{\mathsf{na}} U, V$ and $na_2 : U, V \leq_{\mathsf{na}} X, Y$,*

$$\mathsf{reify}_{\mathsf{na}}\ (\mathsf{mkcomp}_{\mathsf{na}}\ na_1\ na_2) \simeq (\mathsf{reify}_{\mathsf{na}}\ na_1) \cdot (\mathsf{reify}_{\mathsf{na}}\ na_2)$$

.

Now we can prove that reify inverts evaluation.

Lemma 10 (reify-eval). *If $c : A \leq B$, then* reify eval $c \simeq c$.

Proof. By induction on c.

- Case $c = \mathsf{id}_A$: By Prop. 7.
- Case $c = c_1 \cdot c_2$: By Prop. 9 and the IH.
- Case $c =$ mono c: By the IH.
- Case $c =$ arr $c_1\ c_2$: By the IH.
- Case $c =$ fmap: We reason equationally.

$$\mathsf{fmap} \simeq ?$$

[todo: finish proof]

□

Theorem 11. *If $c_1, c_2 : A \leq B$ then $c_1 \simeq c_2$.*

Proof. We reason equationally since $\simeq$ is an equivalence relation. The middle step uses equality, which is finer than $\simeq$ and respected by reify.

$$\begin{aligned} c_1 &\simeq \mathsf{reify}(\mathsf{eval}\ c_1) && \text{by symmetry of } \simeq \text{ and normalization (Lemma 10)} \\ &= \mathsf{reify}(\mathsf{eval}\ c_2) && \text{by uniqueness (Prop 4)} \\ &\simeq c_2 && \text{by normalization (Lemma 10)} \end{aligned}$$

□

6 Implementation in Agda

All of our definitions and theorems are proved in Agda, where we made heavy use of dependently typed syntax and dependent pattern matching to keep our theorems concise and our proofs simple. The presentation of the system in the paper glosses over details that were checked by Agda, like the well-typedness of the output of the reification and evaluation functions.

[todo: key defns here in a syntax-highlighted environment]

We found Agda well suited to the task of mechanizing normalization by evaluation for coercions. All proof assistants are adequate for algebra over a single sort, but in many-sorted and indexed algebras good pattern matching becomes indispensable. Agda's dependent pattern matching allowed

us to use an intrinsically typed syntax and write concise proofs. Recall the multiplication table from Section [todo: ref to mult discussion once it's written]. In Agda, many of the cases we explained away are automatically ruled out by the pattern matching algorithm. In inductive proofs, this makes a big difference in the usability of the system and the efficiency of formalizing metatheory.

While reasoning about coercions, we rely on a user-defined equivalence relation instead of Agda's propositional equality. While Agda has special "rewrite patterns" for rewriting with equations on the left side of a pattern match, we did not find the same level of built-in support for rewriting with custom equivalences. This means that our proofs end up containing big proof terms that are littered with congruence and transitivity rules, like the following clause from the proof of `reify-eval`. What this term really does is a series of rewrites along `c-arr-fmap`, `reify-mkid`, `c-mono-id`, `c-arr-id`, and `c-id-r`, but it also has to declare where in the term those rewrites happen and in what order.

```
reify-eval (c-fmap {a} {b}) =
   c-trans (c-sym c-arr-fmap)
    (c-trans
      (c-cong-comp
        (c-refl c-fmap)
        (c-trans
          (c-cong-arr (c-trans (c-cong-mono (reify-mkid a)) c-mono-id)
                      (c-trans (c-cong-mono (reify-mkid b)) c-mono-id))
          c-arr-id))
      c-id-r)
```

7 Perspectives and future work

We have shown that coercive subtyping for functors is coherent. In the future, we are hoping to extend these results to simplify programming with richer abstractions, like applicative functors and monads. These would permit more sophisticated applications of the subtyping discipline, like in the following example of an evaluator for a calculator language.

```
data Exp = Const Int | Add Exp Exp | Mul Exp Exp | Div Exp Exp

maybeDiv :: Int -> Int -> Maybe Int
maybeDiv n1  0 = Nothing
maybeDiv n1 n2 = Just (div n1 n2)

eval :: Exp -> Maybe Int
eval   (Const n) = n
eval (Add e1 e2) = (+) <$> eval e1 <*> eval e2
eval (Mul e1 e2) = (*) <$> eval e1 <*> eval e2
eval (Div e1 e2) = do n1 <- eval e1
                      n2 <- eval e2
                      maybeDiv n1 n2
```

This program mixes uses of applicative and monad combinators, with applicatives used for total operators and monads for the partial one. Under a subtyping discipline that silently inserts monad and applicative combinators, it could simply be written like this.

```
eval :: Exp -> Maybe Int
eval (Const n) = n
eval (Add e1 e2) = eval e1 + eval e2
```

```
eval (Mul e1 e2) = eval e1 * eval e2
eval (Div e1 e2) = eval e1 `maybeDiv` eval e2
```

Unfortunately, there are a few immediate technical obstacles to achieving this level of brevity in realistic programs while maintaining soundness, decidability, and coherence.

A subtyping system for an applicative functor F should include the declarative coercions pure : $A \leq F\,A$ and seq : $F\,(A \to B) \leq F\,A \to F\,B$. Our normal coercions for plain old functors do not immediately generalize to this setting. The peel invariant (Prop. 3) relied on in some proofs no longer holds because both of the new coercions do not preserve number of leading Fs in types. These coercions introduce many more pairs of coercions that can be composed, and designing a system of normal coercions that can choose a canonical form for all these compositions will require further research.

We have not defined an algorithmic type inference judgment for our system or proved it coherent. We expect this to be straightforward for a monomorphic calculus with just fmap, but there are difficulties in extending the result to polymorphic types. Consider a program that applies `head : [a] -> a` to a list of lists. It already typechecks and it returns the first list. But it is also possible to insert an fmap, which will make it return a list of the head elements of each list. This makes both of the following results plausible, even though they clearly violate coherence.

```
head [[1,2],[3,4]] = [1,2]
head [[1,2],[3,4]] = [1,3]
```

Making this work in general requires incorporating a notion of principal type into the declarative typing judgment so that functions are only lifted when necessary, but formulating this notion will require further work.

As far as proving the system coherent goes, we may be able to scale our method of coherence by normalization to whole programs. While equivalence of programs is undecidable in general, we only need normal forms that are sufficiently organized in order to put two elaborations of the same source term into one shape.

The alternative is to deploy a logical relation in the style of Reynolds [7], whose famous "bracketing theorem" relates an untyped semantics where subtyping is interpreted by inclusion to a typed semantics where subtyping is interpreted by coercion maps. Here there may be a snag. These logical relations relate an untyped semantics, where subtyping is interpreted by inclusion, to a typed semantics, where subtyping is interpreted by coercion maps. In the case of functor subtyping, this requires devising an untyped semantics for programs that makes fmap into an inclusion

$$[\![A \to B]\!] \subseteq [\![F\,A \to F\,B]\!]$$

while still providing a sound model of the rest of the type system. This is an unusual requirement, to say the least, and will require further research.

8 Related Work

Promotiing fmap to a coercion has some similarity with work on effect systems, where programs are permitted to run in an effect bounded below by the effects they actually require. Effect systems may exhibit coherence if liftings between effects do not commute. This issue has been addressed by Biernacki and Polesiuk [4], who use a step-indexed logical relation atop an operational semantics to establish the equivalence of terms typed under different derivations.

The categorical point of view on implicit coercions appears to have originated with Reynolds [6], but we address an issue left unhandled in that paper. He begins with a system of coercions that has at most one coercion between any two types and proceeds to prove that certain term-level operations defined at each type, like addition, commute with the coercions; this establishes the

coherence of terms built from basic operations in the presence of subtyping. Our theorem takes a step back and begins with a possibly incoherent system of declarative coercions. We show our system of normal coercions has at most one coercion between any two types, but we also show that its equational theory matches the equational theory of the original set of coercions. Instead of checking that it commutes with term level operators, we check that it commutes with the operations of the declarative coercion algebra.

Our application of normalization by evaluation to a coherence problem is certainly precedented, although we are unaware of any applications to algorithmic subtyping. We were particularly inspired by the coherence proof for monoidal categories of Beylin and Dyjber [3], which recovers an old result of Mac Lane from a proof of normalization for monoids. In a sense, a monoid is a single-sorted category (or a category is a many-sorted monoid); our normalization of terms involving identities (reflexivity proofs) and compositions (transitivity proofs) bears formal similarities to Beylin and Dyjber's proof.

Allais et al. [1] describe Frex, a suite of tools for working with single-sorted algebraic theories using normalization by evaluation. An extension of Frex to multi-sorted algebraic theories, as was proposed by Allais and an overlapping set of authors at one point [2], would subsume our development and make it easier to add new (non-overlapping) coercions and types to the system without tedious reproving and refactoring.

9 Conclusion

We have presented a subtyping judgment for statically inferring silent uses of the fmap combinator in monomorphic functional programs. Our system uses coercive subtyping, which treats fmap as a coercion that may be inserted wherever the types involved make it make sense. To keep program meaning stable in the face of this potentially intrusive and nondeterministic procedure, we proved coherence of subtyping derivations using a new approach we call *coherence by normalization*. All our definitions and proofs were checked in Agda.

References

[1] Guillaume Allais, Edwin Brady, Nathan Corbyn, Ohad Kammar, and Jeremy Yallop. 2025. Frex: dependently typed algebraic simplification. *Proceedings of the ACM on Programming Languages* 9, ICFP (2025), 30–65.

[2] Guillaume Allais, Nathan Corbyn, Ohad Kammar, Nachiappan Valliappan, Sam Lindley, and Jeremy Yallop. [n. d.]. *Modular construction of multi-sorted free extensions (short paper)*. https://www.cl.cam.ac.uk/~jdy22/papers/modular-construction-of-multi-sorted-free-extensions-short-paper.pdf Presented at PEPM 2023.

[3] Ilya Beylin and Peter Dybjer. 1995. Extracting a proof of coherence for monoidal categories from a proof of normalization for monoids. In *TYPES (Lecture Notes in Computer Science)*. Springer, 47–61.

[4] Dariusz Biernacki and Piotr Polesiuk. 2018. Logical relations for coherence of effect subtyping. *Log. Methods Comput. Sci.* 14, 1 (2018).

[5] Conor McBride and Ross Paterson. 2008. Applicative programming with effects. *Journal of functional programming* 18, 1 (2008), 1–13.

[6] John C. Reynolds. 1980. Using category theory to design implicit conversions and generic operators. In *Semantics-Directed Compiler Generation (Lecture Notes in Computer Science)*. Springer, 211–258.

[7] John C. Reynolds. 2000. *The Meaning of Types: From Intrinsic to Extrinsic Semantics*. Technical Report BRICS RS-00-32. Aarhus University.